

Exhaustive Line-by-Line Code Breakdown: agent/agent_engine.py

Complete Word-by-Word Analysis of Response Synthesis Engine (Lines 1 to 262 — Zero Gaps)

SECTION 1: MODULE DOCSTRING & TYPE ANNOTATIONS (LINES 1–9)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
1	<code>"""</code>	Opening multi-line docstring delimiter marking the beginning of module documentation.
2	<code>Detective Agent - Response Synthesis Engine</code>	Header string declaring the module's core functional identity within the Detective Agent system.
3	<code></code>	Empty line inside the docstring separating title from description.
4	<code>Combines AST/code-structure results and RAG results into</code>	First sentence of description explaining that the module aggregates structural AST and semantic RAG inputs.
5	<code>a readable investigation report.</code>	Second sentence stating the ultimate objective: synthesizing a unified investigation report.
6	<code>"""</code>	Closing multi-line docstring delimiter.
7	<code></code>	Blank line separating module docstring from import statements.
8	<code>from typing import Any, Dict, List</code>	Imports generic type constructs `Any`, `Dict`, and `List` from Python's standard `typing` module for static analysis.
9	<code></code>	Blank line separating imports from helper functions.

SECTION 2: UTILITY FUNCTION `\_SAFE\_TEXT` (LINES 10–23)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
10	<code>def _safe_text(value: Any, default: str = "Not available") -> str:</code>	Defines private helper `_safe_text` accepting any type `value` and optional `default` string, returning a `str`.
11	<code>"""</code>	Opening delimiter for function docstring.
12	<code>Safely converts a value to text.</code>	Summary line describing purpose of conversion.
13	<code></code>	Blank line inside docstring.
14	<code>This prevents None or unexpected values from causing</code>	Explains defensive programming aspect: mitigating `NoneType` errors.
15	<code>formatting problems in the final report.</code>	Completes rationale regarding string formatting safety.
16	<code>"""</code>	Closing docstring delimiter.
17	<code>if value is None:</code>	Evaluates if passed `value` is explicitly `None`.
18	<code>return default</code>	Returns default fallback value immediately if input is `None`.
19	<code></code>	Blank line inside function body.
20	<code>text = str(value).strip()</code>	Converts `value` to string representation and strips leading/trailing whitespace.
21	<code>return text if text else default</code>	Ternary check: returns `text` if non-empty, otherwise returns `default`.
22	<code></code>	Blank line separating functions.
23	<code></code>	Second padding blank line for PEP 8 formatting.

SECTION 3: DICTIONARY FIELD EXTRACTOR `\_GET\_VALUE` (LINES 24–41)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
24	<code>def _get_value(</code>	Function declaration start for `_get_value`.
25	<code>result: Dict[str, Any],</code>	First parameter `result`, expected to be a string-keyed dictionary.
26	<code>key: str,</code>	Second parameter `key`, target dictionary key name as string.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
27	<code>default: str = "Not available",</code>	Third parameter `default`, fallback string if key is absent or invalid.
28	<code>) -> str:</code>	Closing argument list and return type annotation `str`.
29	<code>"""</code>	Opening docstring delimiter.
30	<code>Safely retrieves a value from a result dictionary.</code>	High-level purpose of dictionary lookup utility.
31	<code></code>	Blank line inside docstring.
32	<code>Using .get() instead of result[key] prevents a KeyError</code>	Details protection against `KeyError` exceptions.
33	<code>if the AST parser or RAG system doesn't provide a field.</code>	Explains tolerance for heterogeneous parser and system outputs.
34	<code>"""</code>	Closing docstring delimiter.
35	<code>if not isinstance(result, dict):</code>	Type guard: checks if `result` variable is actually a valid dictionary object.
36	<code>return default</code>	Returns default value immediately if `result` is not a dict.
37	<code></code>	Blank line inside logic block.
38	<code>return _safe_text(result.get(key), default)</code>	Safely retrieves value with `.get(key)` and passes it to `_safe_text` for string cleansing.
39	<code></code>	Blank line separating functions.
40	<code></code>	PEP 8 double newline spacing.
41	<code></code>	Additional padding line.

SECTION 4: RESULT NORMALIZER `\_NORMALISE\_RESULTS` (LINES 42–69)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
42	<code>def _normalise_results(results: Any) -> List[Dict[str, Any]]:</code>	Declares `_normalise_results` taking arbitrary `results` and returning a typed list of dicts.
43	<code>"""</code>	Opening docstring marker.
44	<code>Makes sure parser/RAG output is always handled as a list</code>	Primary requirement: homogenizing input types into list format.
45	<code>of dictionaries.</code>	Continuation of primary requirement statement.
46	<code></code>	Blank docstring line.
47	<code>This allows the application to continue working even if</code>	Explains fault tolerance benefits.
48	<code>a component returns a single dictionary instead of a list,</code>	Handles single-item dictionary return edge case.
49	<code>or returns an unexpected empty value.</code>	Handles `None` or empty container edge case.
50	<code>"""</code>	Closing docstring marker.
51	<code>if not results:</code>	Evaluates falsy values (`None`, `[]`, `{}`, ````).`).
52	<code>return []</code>	Returns empty list for any falsy input.
53	<code></code>	Blank line.
54	<code>if isinstance(results, dict):</code>	Checks if single dictionary was provided.
55	<code>results = [results]</code>	Wraps single dictionary into a 1-element list.
56	<code></code>	Blank line.
57	<code>if not isinstance(results, (list, tuple)):</code>	Ensures input is now either a `list` or a `tuple`.
58	<code>return []</code>	Returns empty list if input is invalid type (e.g. integer or set).
59	<code></code>	Blank line.
60	<code>return [</code>	Starts list comprehension block for filtering elements.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
61	<code>result</code>	Element to yield if filter condition passes.
62	<code>for result in results</code>	Iterates through items in `results` container.
63	<code>if isinstance(result, dict)</code>	Filter condition: keeps element only if it is a `dict`.
64	<code>]</code>	Closes list comprehension.
65		Blank line.
66		Blank line spacing.
67–69		PEP 8 compliance blank lines.

SECTION 5: RESULT DEDUPLICATOR `\_DEDUPLICATE\_RESULTS` (LINES 70–108)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
70	<code>def _deduplicate_results(</code>	Function declaration start for deduplication logic.
71	<code>results: List[Dict[str, Any]]</code>	Parameter typed as a list of dictionaries.
72	<code>) -> List[Dict[str, Any]]:</code>	Return type signature matching input list structure.
73	<code>"""</code>	Docstring opening.
74	<code>Removes duplicate evidence while preserving its original order.</code>	States order-preserving deduplication property.
75		Blank line.
76	<code>This keeps the Detective Agent report cleaner when the same</code>	Explains report readability benefits.
77	<code>code evidence is returned more than once by a search component.</code>	Describes multiple-retrieval overlap scenario.
78	<code>"""</code>	Docstring closing.
79	<code>unique = []</code>	Initializes empty list to accumulate ordered unique result dicts.
80	<code>seen = set()</code>	Initializes empty hash set for O(1) lookup of existing fingerprints.
81		Blank line.
82	<code>for result in results:</code>	Iterates sequentially through normalized input result list.
83	<code>path = _get_value(result, "path", "")</code>	Safely extracts `path` field, defaulting to empty string.
84	<code>details = _get_value(result, "details", "")</code>	Safely extracts `details` field, defaulting to empty string.
85	<code>snippet = _get_value(result, "snippet", "")</code>	Safely extracts `snippet` field, defaulting to empty string.
86		Blank line.
87	<code># A lightweight fingerprint is created from the available</code>	Inline comment explaining fingerprint construction concept.
88	<code># evidence so identical results can be detected.</code>	Comment continuation detailing duplicate detection logic.
89	<code>fingerprint = (</code>	Starts tuple definition for unique hashable fingerprint key.
90	<code>path.lower(),</code>	Case-insensitive lowercased path string.
91	<code>details.lower(),</code>	Case-insensitive lowercased details string.
92	<code>snippet.lower(),</code>	Case-insensitive lowercased snippet string.
93	<code>)</code>	Closes fingerprint tuple creation.
94		Blank line.
95	<code>if fingerprint in seen:</code>	Checks if constructed fingerprint already exists in `seen` set.
96	<code>continue</code>	Skips current item iteration if fingerprint is a duplicate.
97		Blank line.
98	<code>seen.add(fingerprint)</code>	Adds new unique fingerprint tuple to tracking set.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
99	<code>unique.append(result)</code>	Appends unique result dictionary to output ordering list.
100		Blank line.
101	<code>return unique</code>	Returns deduplicated list maintaining original occurrence order.
102–108		Blank spacing lines separating helper functions.

SECTION 6: CROSS-SYSTEM CORRELATION `\_FIND\_RELATED\_FILES` (LINES 109–141)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
109	<code>def _find_related_files(</code>	Function declaration for identifying file overlaps across systems.
110	<code>ast_results: List[Dict[str, Any]],</code>	First parameter: list of normalized AST result dictionaries.
111	<code>rag_results: List[Dict[str, Any]],</code>	Second parameter: list of normalized RAG result dictionaries.
112	<code>) -> List[str]:</code>	Return type annotation specifying a list of file path strings.
113	<code>"""</code>	Docstring start marker.
114	<code> Finds files that appear in both AST and RAG results.</code>	Summarizes cross-check intent.
115		Blank docstring line.
116	<code> This provides a simple cross-check between structural code</code>	Highlights cross-verification between AST structure
117	<code>analysis and semantic retrieval.</code>	and RAG semantic search.
118	<code>"""</code>	Docstring end marker.
119	<code>ast_files = {</code>	Starts set comprehension to gather all unique file paths in AST data.
120	<code> _get_value(result, "path", "")</code>	Extracts path value for each AST result dict.
121	<code>for result in ast_results</code>	Loops over items in AST result list.
122	<code> if _get_value(result, "path", "") != "Not available"</code>	Filters out entries missing a valid path attribute.
123	<code>}</code>	Closes AST file set comprehension.
124		Blank line.
125	<code>rag_files = {</code>	Starts set comprehension to gather all unique file paths in RAG data.
126	<code> _get_value(result, "path", "")</code>	Extracts path value for each RAG result dict.
127	<code>for result in rag_results</code>	Loops over items in RAG result list.
128	<code> if _get_value(result, "path", "") != "Not available"</code>	Filters out entries missing a valid path attribute.
129	<code>}</code>	Closes RAG file set comprehension.
130		Blank line.
131	<code>return sorted(ast_files.intersection(rag_files))</code>	Calculates mathematical set intersection of paths and returns them alphabetically sorted.
132–141		Blank spacing lines separating utility functions from public orchestrator.

SECTION 7: MAIN SYNTHESIS ORCHESTRATOR `GENERATE\_AGENT\_RESPONSE` (LINES 142–185)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
142	<code>def generate_agent_response(query, ast_results, rag_results):</code>	Main public function signature accepting user query, AST findings, and RAG findings.
143	<code>"""</code>	Docstring opening marker.
144	<code> Combines AST parser results and RAG results into the</code>	Describes final report aggregation job.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
145	<code>Detective Agent's final response.</code>	Continuation of task description.
146	<code></code>	Blank docstring line.
147	<code>The function signature is intentionally kept unchanged</code>	Notes strict backward compatibility guarantee for public signature.
148	<code>so existing code calling this function continues to work.</code>	Explains API stability rationale.
149	<code>"""</code>	Docstring closing marker.
150	<code></code>	Blank line.
151	<code># Convert incoming results into predictable formats before</code>	Comment line explaining pre-processing steps.
152	<code># processing them. This prevents malformed/empty results</code>	Comment on crash prevention.
153	<code># from crashing the entire application.</code>	Comment continuation.
154	<code>ast_results = _normalise_results(ast_results)</code>	Normalizes `ast_results` input into valid list of dicts.
155	<code>rag_results = _normalise_results(rag_results)</code>	Normalizes `rag_results` input into valid list of dicts.
156	<code></code>	Blank line.
157	<code># Remove repeated evidence so the final report does n't</code>	Comment on deduplication objective.
158	<code># contain unnecessary duplicate information.</code>	Comment continuation.
159	<code>ast_results = _deduplicate_results(ast_results)</code>	Deduplicates AST result entries in-place.
160	<code>rag_results = _deduplicate_results(rag_results)</code>	Deduplicates RAG result entries in-place.
161	<code></code>	Blank line.
162	<code>query_text = _safe_text(query, "No query provided")</code>	Cleans user query string or sets fallback text.
163	<code></code>	Blank line.
164	<code># Preserve the original behavior for a search where neither</code>	Comment regarding empty result set fallback.
165	<code># the AST parser nor RAG system found anything.</code>	Comment continuation.
166	<code>if not ast_results and not rag_results:</code>	Condition checking if both evidence lists are completely empty.
167	<code>return (</code>	Returns formatted "no findings" string tuple concatenation start.
168	<code>"🕵️ *Detective Agent:* "</code>	Adds agent identity icon prefix.
169	<code>f"I searched the codebase for *{query_text}*, "</code>	Interpolates query text into message.
170	<code>"but found no structural definitions or matching "</code>	States lack of structural match.
171	<code>"code evidence."</code>	States lack of semantic code evidence match.
172	<code>)</code>	Closes return statement tuple.
173	<code></code>	Blank line.
174	<code>report = [</code>	Initializes report block list with top header string.
175	<code>f"🕵️ *Detective Agent Analysis for:* {query_text} \n"</code>	Creates bold main title line containing user query text.
176	<code>]</code>	Closes report list initialization.
177–185	<code></code>	Blank line section dividers.

SECTION 8: AST SECTION FORMATTING & RAG FORMATTING (LINES 186–220)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
186	<pre># ----- -----</pre>	Visual section divider comment for AST section.
187	<pre># AST / CODE STRUCTURE RESULTS</pre>	Comment labeling AST section.
188	<pre># ----- -----</pre>	Closing visual divider comment line.
189	<pre></pre>	Blank line.
190	<pre>if ast_results:</pre>	Executes AST formatting block if AST findings exist.
191	<pre>report.append("### 🔍 Code Structure Summary")</pre>	Appends section header for structural evidence.
192	<pre></pre>	Blank line.
193	<pre>for result in ast_results:</pre>	Loops through each deduplicated AST result item.
194	<pre># These values are retrieved safely so that a missi ng</pre>	Comment on safe property access.
195	<pre># field doesn't terminate the whole investigation.</pre>	Comment continuation on error tolerance.
196	<pre>result_type = _get_value(result, "type")</pre>	Safely extracts construct type (e.g., class, function).
197	<pre>details = _get_value(result, "details")</pre>	Safely extracts implementation details string.
198	<pre>path = _get_value(result, "path")</pre>	Safely extracts file path string.
199	<pre></pre>	Blank line.
200	<pre>report.append(f"- *Type:* {result_type.upper()}\n" f"- *Details:* {details}\n" f"- *File:* {path}")</pre>	Appends bulleted item block for current AST item.
201		Formats upper-cased code construct type line.
202		Formats details description line.
203		Formats file location line.
204		Closes report append call.
205	<pre></pre>	Blank line.
206	<pre># ----- -----</pre>	Visual section divider comment for RAG section.
207	<pre># RAG RESULTS</pre>	Comment labeling RAG section.
208	<pre># ----- -----</pre>	Closing visual divider comment line.
209	<pre></pre>	Blank line.
210	<pre>if rag_results:</pre>	Executes RAG formatting block if semantic RAG findings exist.
211	<pre>report.append("\n### 📖 Relevant Code Snippets")</pre>	Appends section header for RAG code snippets.
212	<pre></pre>	Blank line.
213	<pre>for result in rag_results:</pre>	Loops through each deduplicated RAG item.
214	<pre>path = _get_value(result, "path")</pre>	Safely extracts file path string for RAG snippet.
215	<pre>snippet = _get_value(result, "snippet")</pre>	Safely extracts actual code snippet text.
216	<pre></pre>	Blank line.
217	<pre>report.append(f"- *File:* {path}\n" f"- *Context:* \n" f"text\n{snippet}\n"</pre>	Appends formatted snippet block to report.
218		Formats file path reference.
219		Formats context heading indicator.
220		Embeds code snippet inside formatted text block.

SECTION 9: CROSS-CHECK & CORRELATION FORMATTING (LINES 221–250)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
221	<pre>)</pre>	Closes report append call for RAG result item.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
222		Blank line.
223	<pre># ----- -----</pre>	Visual section divider comment for cross-check block.
224	<pre># AST + RAG CROSS-CHECK</pre>	Comment labeling cross-correlation section.
225	<pre># ----- -----</pre>	Closing visual divider comment line.
226		Blank line.
227	<pre># A file found by both systems provides stronger ev idence</pre>	Comment explaining dual-system validation logic.
228	<pre># than a file found by only one source.</pre>	Comment continuation on evidence weighting.
229	<pre>related_files = _find_related_files(</pre>	Invokes `_find_related_files` passing AST and RAG lists.
230	<pre>ast_results,</pre>	First argument: normalized AST results.
231	<pre>rag_results,</pre>	Second argument: normalized RAG results.
232	<pre>)</pre>	Closes `_find_related_files` call.
233		Blank line.
234	<pre>if related_files:</pre>	Checks if any overlapping file paths were discovered.
235	<pre>report.append("\n### 🔗 Correlated Evidence")</pre>	Appends header for dual-verified correlated evidence.
236		Blank line.
237	<pre>report.append(</pre>	Appends explanatory sentence for correlation findings.
238	<pre>"The following files were identified by both "</pre>	First part of correlation explanation.
239	<pre>"code-structure analysis and RAG retrieval:"</pre>	Second part of correlation explanation.
240	<pre>)</pre>	Closes report append call.
241		Blank line.
242	<pre>for path in related_files:</pre>	Iterates through list of overlapping file paths.
243	<pre>report.append(f"- {path}")</pre>	Appends bulleted path entry to report list.
244		Blank line.
245	<pre># ----- -----</pre>	Visual section divider comment for investigation summary.
246	<pre># INVESTIGATION SUMMARY</pre>	Comment labeling summary block.
247	<pre># ----- -----</pre>	Closing visual divider comment line.
248		Blank line.
249	<pre>evidence_sources = []</pre>	Initializes container list for high-level source counts.
250		Blank line.

SECTION 10: INVESTIGATION SUMMARY & FINAL RETURN (LINES 251–262)

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
251	<pre>if ast_results:</pre>	Checks if AST results were present to build summary string.
252	<pre>evidence_sources.append(</pre>	Appends formatted count string for AST findings.
253	<pre>f"{len(ast_results)} structural result(s)"</pre>	Interpolates length of `ast_results` list.
254	<pre>)</pre>	Closes append call.
255		Blank line.
256	<pre>if rag_results:</pre>	Checks if RAG results were present to build summary string.
257	<pre>evidence_sources.append(</pre>	Appends formatted count string for RAG findings.

Line	Exact Source Code	Word-by-Word & Line-by-Line Explanation
258	<code>f"{len(rag_results)} retrieved snippet(s)"</code>	Interpolates length of `rag_results` list.
259	<code>)</code>	Closes append call.
260	<code></code>	Blank line.
261	<code>report.append("\n### 🧠 Investigation Summary")</code>	Appends section title for final investigation summary block.
262	<code>return "\n\n".join(report)</code>	Joins all report sections using double newlines and returns completed Markdown string.